UNIVERSIDADE DE VILA VELHA

PROJETO E ANÁLISE DE ALGORITMOS

Explorando a intratabilidade e a força bruta no Problema do Caixeiro Viajante

Bernardo Ziviani Gasparino
Samuel Bruno da Silva
Yuri Luiz Kruger Torquato

Professor: Wanderson Muniz de Santana

Turma: CC7N

Vila Velha - ES
2026

Sumário

1 Introdução **2**

2 Fundamentação teórica **2**
2.1 Modelagem e força bruta 2
2.2 Fixação da origem e redução do espaço de busca 2

3 Implementação **2**
3.1 Algoritmo sequencial 2
3.2 Implementação paralela em GPU 3

4 Metodologia experimental **3**
4.1 Ambiente e configuração 3
4.2 Procedimento de execução 4

5 Resultados e validação **4**
5.1 Validação em uma instância de referência 4
5.2 Tabela de crescimento combinatório 5
5.3 Curva teórica e dados empíricos 6
5.4 Evidências de execução 8

6 Discussão das questões investigativas **9**
6.1 Redução do espaço de busca 9
6.2 Medições na CPU sem fixar a origem 11
6.3 Custo de avaliação por rota 12
6.4 Limiar prático da intratabilidade 12
6.5 Classes P e NP e heurísticas 12

7 Conclusão **13**

A Tempos individuais e dispersão **14**
A.1 Medições individuais: CPU, origem fixa 14
A.2 Medições individuais: GPU, origem fixa 15
A.3 Medições individuais: CPU, todas as origens 16

1 Introdução

O Problema do Caixeiro Viajante, conhecido pela sigla TSP, consiste em encontrar um circuito de custo mínimo que visite cada cidade uma única vez e retorne à cidade de origem. Embora sua definição seja simples, a quantidade de rotas candidatas cresce rapidamente com o número de cidades. Este trabalho investiga esse crescimento por meio de uma implementação de força bruta, relacionando as medições experimentais à análise apresentada no material da disciplina [1] e aos objetivos da proposta de atividade [2].

O objetivo é implementar a busca exaustiva, registrar as quantidades de rotas e avaliações de arestas, medir o tempo para diferentes tamanhos de entrada e comparar os resultados com uma curva proporcional a $n!$.

2 Fundamentação teórica

2.1 Modelagem e força bruta

O conjunto de cidades é representado por $V = \{v_1, v_2, \ldots, v_n\}$, e o custo de deslocamento entre duas cidades é dado por $d(v_i, v_j)$. Na implementação, os custos são armazenados em uma matriz. Para cada rota candidata, o programa soma os custos das arestas percorridas e mantém a rota de menor custo encontrada.

Para evitar contar diferentes rotações do mesmo circuito, fixa-se v_1 como origem na execução principal. Restam, portanto, $n-1$ cidades a serem permutadas, resultando em

$$R(n) = (n-1)! \tag{1}$$

rotas. Cada rota possui n arestas, incluindo o retorno à origem. O número total de avaliações dessas arestas é

$$A(n) = n \cdot (n-1)! = n!. \tag{2}$$

Assim, para a implementação sequencial com avaliação linear por rota, o tempo de execução é $\Theta(n!)$ [1]. A igualdade acima representa o número de avaliações de arestas, não a quantidade total de instruções executadas pelo programa.

Como a matriz gerada neste projeto é simétrica, uma rota e sua inversa têm o mesmo custo. Mesmo assim, ambas são avaliadas separadamente, preservando a contagem $(n-1)!$ utilizada no material. Para $n = 6$, por exemplo, são $5! = 120$ rotas e $6 \cdot 120 = 720$ avaliações de arestas.

2.2 Fixação da origem e redução do espaço de busca

Sem fixar a origem, a enumeração de todas as permutações das n cidades produziria $n!$ representações. Ao fixá-la, o total é reduzido para $(n-1)!$. Como $n! = n \cdot (n-1)!$, a redução é por um fator igual a n. As diferentes escolhas de origem correspondem a rotações de um mesmo circuito e não alteram seu custo ótimo.

3 Implementação

3.1 Algoritmo sequencial

A implementação em Go gera uma matriz simétrica de custos positivos a partir de uma semente e enumera recursivamente as permutações das cidades restantes. Para cada

permutação, constrói o circuito, avalia suas n arestas e atualiza a melhor solução quando encontra um custo inferior. São registrados a rota ótima, seu custo, as contagens e o tempo de busca [3].

3.2 Implementação paralela em GPU

Além da versão sequencial, o projeto possui uma implementação em CUDA, integrada ao programa em Go por meio de CGO. A matriz de custos é preparada no hospedeiro e copiada para a memória da GPU. A enumeração das $(n-1)!$ permutações é distribuída entre as threads CUDA: cada thread calcula custos de rotas e participa da redução do menor custo. Como uma redução paralela identifica o custo mínimo, mas não necessariamente a rota que o produziu, uma segunda passagem procura o índice da primeira rota com esse custo. A partir desse índice, o programa reconstrói a rota ótima e devolve o resultado ao Go.

O tempo da versão GPU registrado pelo programa inclui a chamada à rotina CUDA, alocação e transferência de dados, as duas passagens de busca e a sincronização necessária para obter os resultados no hospedeiro. Portanto, especialmente para entradas pequenas, o custo fixo de inicialização e transferência pode ser maior que o tempo efetivamente gasto calculando rotas.

4 Metodologia experimental

4.1 Ambiente e configuração

Os experimentos foram executados em computadores distintos para CPU e GPU. As configurações informadas são:

- CPU (notebook): AMD Ryzen 7 4800H, 20 GB de RAM e EndeavourOS (baseado em Arch Linux);
- GPU: AMD Ryzen 7 5700G, 32 GB de RAM e NVIDIA GeForce RTX 5060 Ti com 16 GB, usando CachyOS;
- Go 1.27.3 nas execuções; CUDA 13.4.92 na execução GPU;
- CPU: $n = 3$ a $n = 15$; GPU: $n = 3$ a $n = 17$; origem fixa na cidade 1 e semente 42.

O tempo registrado compreende a enumeração, a montagem e a avaliação das rotas, incluindo a instrumentação. Na GPU inclui as duas passagens. A geração da matriz e a gravação dos resultados não fazem parte desse intervalo. O registro detalhado das rotas não foi considerado na medição de desempenho.

Para cada tamanho n, o programa enumera as rotas e registra o tempo de busca. Quando existem várias execuções para o mesmo tamanho e dispositivo, os tempos são resumidos pela mediana: ordenam-se as amostras e toma-se o valor central (ou a média dos dois centrais, se a quantidade for par). Foram consideradas seis execuções de CPU para $n = 3$ a $n = 10$, entre quatro e cinco para $n = 11$ a $n = 14$, e uma para $n = 15$. Na GPU, há duas amostras para $n = 3$ a $n = 16$ e uma para $n = 17$. Portanto, os resultados para $n = 15$ na CPU e $n = 17$ na GPU são medições únicas, não medianas robustas.

O Apêndice A apresenta os tempos individuais e os valores mínimo, mediano e máximo para cada tamanho. Os intervalos descrevem a variação observada, não um intervalo de

confiança. Não foi possível confirmar que todas as execuções ocorreram sob condições idênticas. Além disso, matrizes geradas com a mesma semente não são necessariamente iguais quando se altera o tamanho máximo utilizado na geração. Para uma comparação controlada, devem-se usar as mesmas instâncias e manter constantes as condições de execução.

4.2 Procedimento de execução

As buscas principais com origem fixa podem ser executadas com os parâmetros abaixo. O primeiro comando utiliza a versão sequencial e o segundo utiliza a GPU, que requer uma GPU NVIDIA e o ambiente CUDA configurado:

```
make run ARGS="-min-n 3 -max-n 15 -seed 42"
make gurun ARGS="-min-n 3 -max-n 17 -seed 42"
```

Cada comando realiza uma execução por tamanho; repetições independentes são necessárias para calcular medianas. Os maiores casos podem exigir muitas horas. Esses comandos reproduzem os parâmetros informados, mas não garantem tempos idênticos nem as mesmas condições das medições anteriores.

Para comparar as duas formas de enumeração em uma mesma instância por tamanho, foram usados os comandos abaixo, ambos com semente 42 e tamanho máximo 8:

```
make run ARGS="-min-n 3 -max-n 8 -seed 42"
make run-random ARGS="-min-n 3 -max-n 8 -seed 42"
```

No primeiro caso, a origem permanece fixa. No segundo, apesar do nome do comando, a busca é executada a partir de todas as cidades, enumerando também as rotações redundantes. A comparação correspondente aparece na Seção 6.1.

Também foram realizadas execuções sequenciais sem fixação, com origem em cada uma das cidades, usando semente 42 e limites máximos de entrada variados. Para cada tamanho, o tempo corresponde à soma dos tempos das buscas individuais a partir de cada origem. Uma execução interrompida contribui apenas com os tamanhos já concluídos. Como o tamanho máximo influencia a matriz gerada, as amostras de um mesmo n não necessariamente correspondem à mesma instância. As medianas resumem os tempos observados nessas condições, e não repetidas medições necessariamente da mesma matriz.

5 Resultados e validação

5.1 Validação em uma instância de referência

Para verificar o custo mínimo, considerou-se uma instância simétrica de quatro cidades, com a seguinte matriz de custos:

$$D = \begin{pmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{pmatrix}.$$

Fixando a cidade 1, as seis rotas possíveis têm os custos abaixo, obtidos pela soma das quatro arestas de cada circuito:

Rota	Custo	Rota	Custo
$1 \to 2 \to 3 \to 4 \to 1$	95	$1 \to 2 \to 4 \to 3 \to 1$	80
$1 \to 3 \to 2 \to 4 \to 1$	95	$1 \to 3 \to 4 \to 2 \to 1$	80
$1 \to 4 \to 2 \to 3 \to 1$	95	$1 \to 4 \to 3 \to 2 \to 1$	95

O menor custo de referência é, portanto, 80, obtido por duas rotas inversas. Na execução do teste sequencial com essa matriz, a implementação também retornou custo 80, contabilizando seis rotas e 24 avaliações de arestas. A concordância valida esse caso específico, embora não prove, isoladamente, a correção para todas as entradas.

5.2 Tabela de crescimento combinatório

A Tabela 1 e a Tabela 2 apresentam as contagens teóricas por passagem e os tempos resumidos por mediana, em milissegundos. A comparação com os registros confirmou, em todas as entradas, $(n-1)!$ rotas e $n!$ avaliações de arestas na CPU e na primeira passagem da GPU. Por exemplo, para $n = 10$ foram registradas 362.880 rotas e 3.628.800 avaliações de arestas. Nas medições apresentadas, as contagens da GPU abrangem apenas a primeira passagem, embora o tempo inclua duas. A segunda passagem reavalia todas as rotas; portanto, o total efetivo das duas passagens é $2(n-1)!$ avaliações de rotas e $2n!$ avaliações de arestas. Os tempos resumem as amostras disponíveis; os valores individuais constam no apêndice.

Tabela 1: Resultados da execução em CPU: contagens teóricas e mediana do tempo.

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
3	2	6	6	0,000922
4	6	24	6	0,000547
5	24	120	6	0,001182
6	120	720	6	0,005270
7	720	5.040	6	0,031848
8	5.040	40.320	6	0,323821
9	40.320	362.880	6	3,124497
10	362.880	3.628.800	6	22,118782
11	3.628.800	39.916.800	5	234,589688
12	39.916.800	479.001.600	5	2.675,783333
13	479.001.600	6.227.020.800	4	31.836,321137
14	6.227.020.800	87.178.291.200	4	433.055,072678
15	87.178.291.200	1.307.674.368.000	1	5.981.114,958103

Tabela 2: Resultados da execução em GPU: contagens teóricas e mediana do tempo.

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
3	2	6	2	151,951220
4	6	24	2	0,211461

n	$(n-1)!$	$n!$	Amostras	Mediana (ms)
5	24	120	2	0,737821
6	120	720	2	0,348593
7	720	5.040	2	0,184059
8	5.040	40.320	2	0,283575
9	40.320	362.880	2	0,229065
10	362.880	3.628.800	2	0,571896
11	3.628.800	39.916.800	2	3,330259
12	39.916.800	479.001.600	2	32,461620
13	479.001.600	6.227.020.800	2	402,219234
14	6.227.020.800	87.178.291.200	2	6.073,329214
15	87.178.291.200	1.307.674.368.000	2	104.752,111644
16	1.307.674.368.000	20.922.789.888.000	2	1.592.764,836502
17	20.922.789.888.000	355.687.428.096.000	1	27.074.814,512025

5.3 Curva teórica e dados empíricos

Cada gráfico sobrepõe o tempo medido à função $f(n) = c \cdot n!$. A constante c é ajustada pelo método dos mínimos quadrados aos tempos em milissegundos, considerando uma reta que passa pela origem. Nos gráficos agregados, t_i é a mediana das amostras disponíveis para o tamanho n_i; quando existe apenas uma amostra, a mediana coincide com esse único valor. Para os pares $(x_i, y_i) = (n_i!, t_i)$, o valor utilizado é

$$c = \frac{\sum_i x_i y_i}{\sum_i x_i^2}. \tag{3}$$

O ajuste é realizado sobre os tempos em escala linear; consequentemente, as maiores medições têm maior influência. O gráfico utiliza escala logarítmica no eixo vertical apenas para facilitar a visualização de valores de ordens de grandeza diferentes. A proximidade entre as curvas ajuda na comparação, mas não constitui, isoladamente, uma prova da complexidade assintótica.

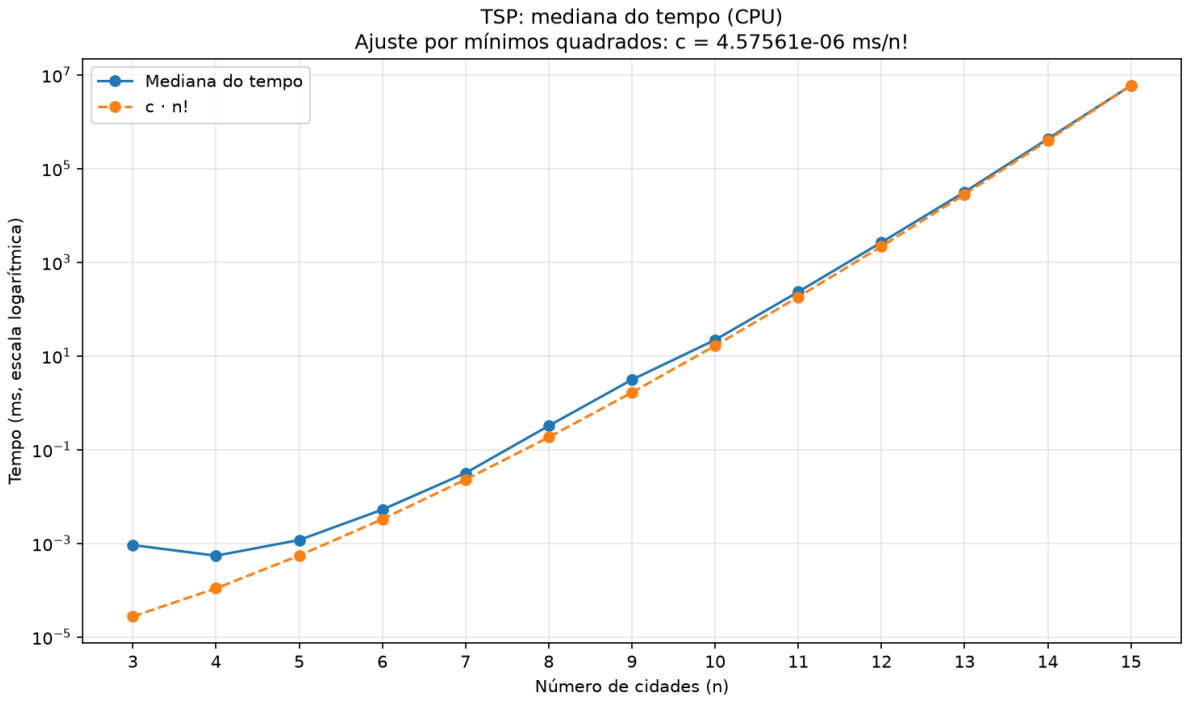


Figura 1: Mediana dos tempos da CPU e ajuste fatorial.

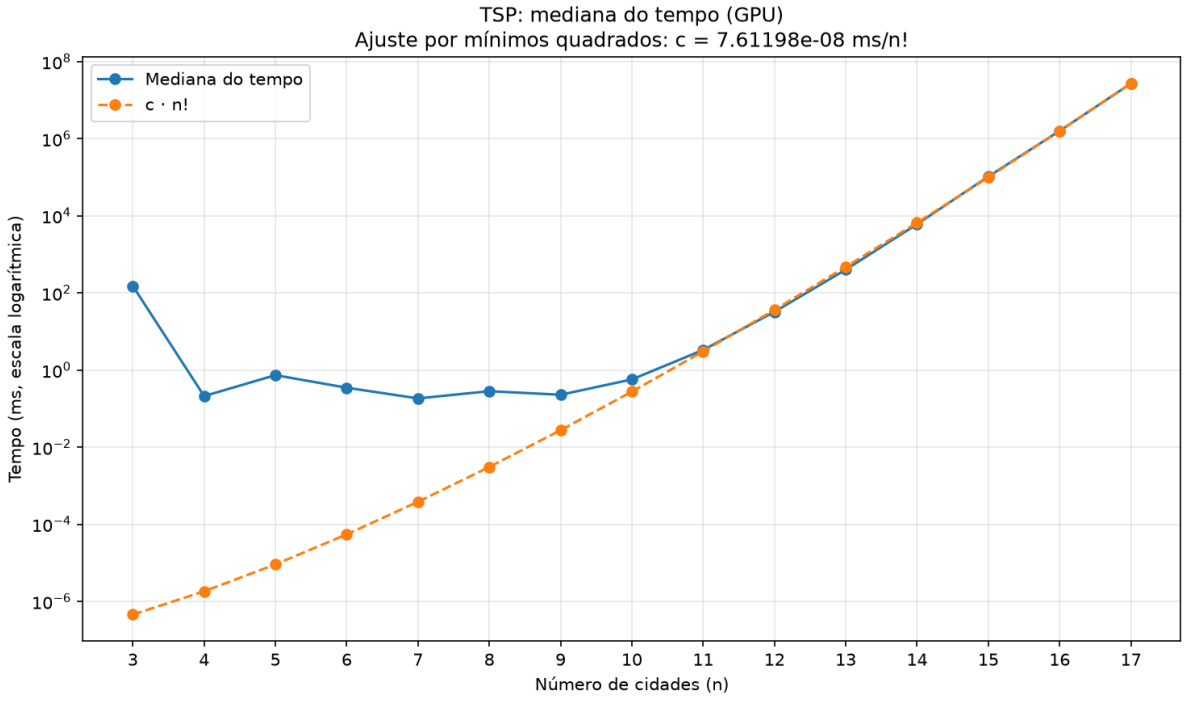


Figura 2: Mediana dos tempos da GPU e ajuste fatorial.

Na execução em CPU (Figura 1), o gráfico abrange $n = 3$ a $n = 15$ e o ajuste às medianas informa $c = 4{,}57561 \times 10^{-6}$ ms/$n!$. Na execução em GPU (Figura 2), o intervalo é $n = 3$ a $n = 17$ e o ajuste informa $c = 7{,}61198 \times 10^{-8}$ ms/$n!$. Esses coeficientes são ajustes independentes, calculados sobre os dados de cada execução, e não constituem

uma razão de desempenho entre CPU e GPU.

Como exemplos concretos da tabela, em $n = 10$ a mediana foi de 22,118782 ms na CPU (seis amostras) e 0,571896 ms na GPU (duas amostras). Na CPU, $n = 14$ levou em mediana 433.055,072678 ms, aproximadamente 7 min 13 s, considerando quatro amostras. O maior caso concluído na GPU, $n = 17$, levou 27.074.814,512025 ms, aproximadamente 7 h 31 min, mas esse valor veio de uma única execução. A mediana de uma só amostra é apenas o tempo observado naquela execução.

As medições foram realizadas em computadores diferentes, portanto os gráficos não isolam o efeito do dispositivo: processadores, memória e sistemas operacionais também diferem. O tempo para $n = 3$ na GPU é alto em relação aos tamanhos seguintes, compatível com o custo fixo de inicialização e transferência CUDA. Para comparação controlada de desempenho, seria necessário executar ambas as versões na mesma máquina, com as mesmas instâncias e várias repetições. O ajuste fatorial descreve o crescimento; não demonstra, por si só, que os tempos de CPU e GPU sejam comparáveis.

5.4 Evidências de execução

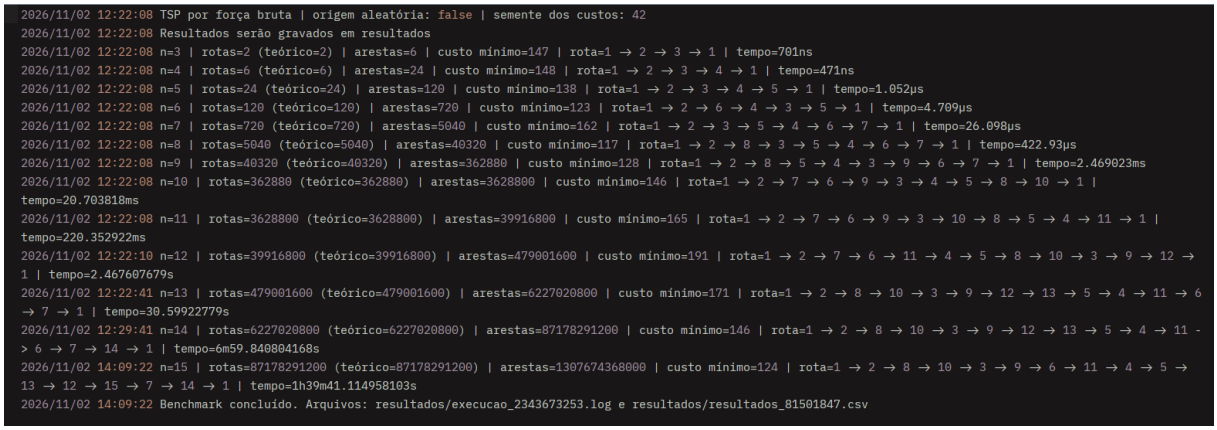


Figura 3: Saída da execução CPU, incluindo as contagens e tempos registrados.

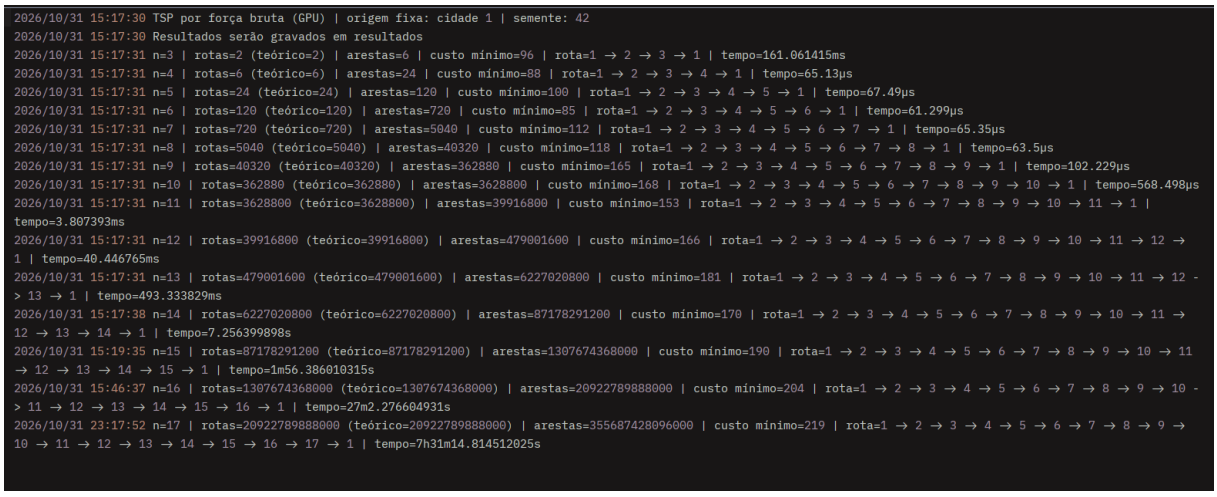


Figura 4: Saída da execução GPU, incluindo as contagens e tempos registrados.

6 Discussão das questões investigativas

6.1 Redução do espaço de busca

Sem fixar a cidade inicial, seriam enumeradas $n!$ permutações. Como cada circuito pode ser escrito começando por qualquer uma das suas n cidades, essas permutações incluem n rotações do mesmo circuito. Fixar v_1 elimina essa redundância e deixa $(n-1)!$ rotas, reduzindo por um fator n. Como cada rota exige avaliar n arestas, sem a fixação seriam feitas $n \cdot n!$ avaliações de arestas; com a fixação, são $n \cdot (n-1)! = n!$.

O código reflete essa redução: tanto a versão sequencial quanto a GPU mantêm a cidade de início fixa (cidade 1 nos testes principais) e permutam apenas as demais. Para $n = 10$, por exemplo, a CPU e cada passagem da GPU avaliaram $9! = 362.880$ rotas e $10! = 3.628.800$ arestas. Sem fixar a origem, seriam $10! = 3.628.800$ representações de rotas e $10 \cdot 10! = 36.288.000$ avaliações de arestas por passagem, sem obter novos circuitos distintos nem melhorar o custo ótimo. As contagens históricas observadas coincidem com os valores teóricos por passagem para todos os tamanhos testados.

Para verificar também o caso sem fixação, foram realizadas duas execuções sequenciais adicionais com a mesma matriz gerada para $n = 4$ e semente 42. Na primeira, fixou-se a cidade 1; na segunda, enumeraram-se as rotas a partir de cada uma das quatro cidades. Os resultados observados foram:

Configuração	Rotas avaliadas	Arestas avaliadas	Custo mínimo
Origem fixa	6	24	164
Todas as origens	24	96	164

Nesse experimento, não fixar a origem multiplicou por quatro as contagens, preservando o custo mínimo, como previsto. Esses valores correspondem a uma instância própria, distinta da matriz de referência da seção anterior; não representam uma medição na GPU.

Em uma segunda comparação sequencial, foram utilizadas as mesmas instâncias nos dois modos para cada n de 3 a 8. A tabela apresenta as contagens efetivamente observadas e os custos mínimos. Em todos os casos, os dois modos encontraram o mesmo custo ótimo.

Tabela 3: Comparação experimental na CPU, com e sem origem fixa.

n	Rotas fixas	Todas as rotas	Arestas fixas	Todas as arestas	Custo ótimo
3	2	6	6	18	171
4	6	24	24	96	152
5	24	120	120	600	168
6	120	720	720	4.320	161
7	720	5.040	5.040	35.280	228
8	5.040	40.320	40.320	322.560	195

Para $n = 8$, por exemplo, a rota ótima encontrada foi $1 \to 2 \to 5 \to 4 \to 6 \to 8 \to 3 \to 7 \to 1$, com custo 195. A Figura 5 representa as contagens e os custos encontrados; as curvas de custo coincidem porque as instâncias são as mesmas. Não se espera que o custo ótimo cresça monotonamente entre tamanhos, pois as instâncias possuem custos diferentes.

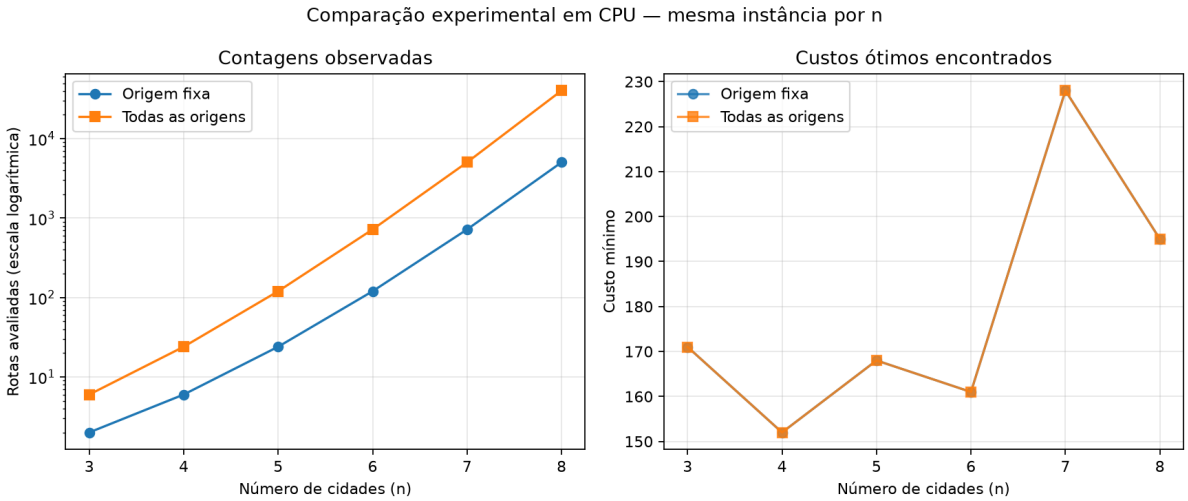


Figura 5: Contagens de rotas e custos ótimos observados nas duas configurações.

Tabela 4: Comparação teórica do espaço de busca com e sem origem fixa.

n	Rotas fixas	Rotas sem fixar	Arestas fixas	Arestas sem fixar
3	2	6	6	18
6	120	720	720	4.320
10	362.880	3.628.800	3.628.800	36.288.000
15	$8{,}72 \times 10^{10}$	$1{,}31 \times 10^{12}$	$1{,}31 \times 10^{12}$	$1{,}96 \times 10^{13}$
17	$2{,}09 \times 10^{13}$	$3{,}56 \times 10^{14}$	$3{,}56 \times 10^{14}$	$6{,}05 \times 10^{15}$

Os valores da Tabela 4 são calculados pelas fórmulas por passagem; não representam medições sem fixação. A Figura 6 também é teórica. Ao enumerar todas as origens, seriam avaliadas $n!$ rotas e $n \cdot n!$ arestas em uma passagem; em duas passagens, seriam $2n!$ avaliações de rotas e $2n \cdot n!$ avaliações de arestas.

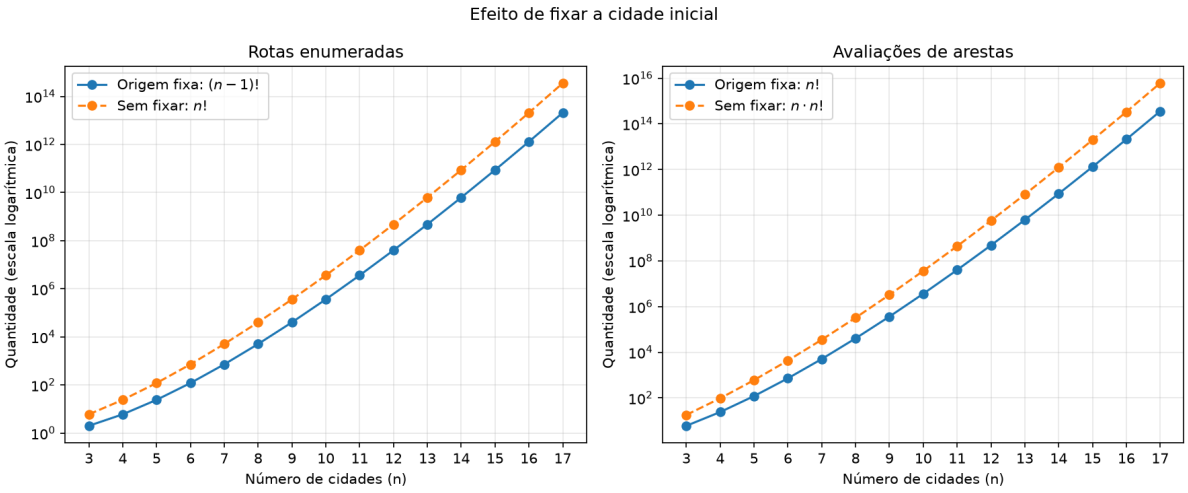


Figura 6: Quantidade teórica de rotas e avaliações de arestas com origem fixa e sem fixá-la.

6.2 Medições na CPU sem fixar a origem

A Tabela 5 reúne os resultados das execuções sequenciais que enumeraram todas as origens. As contagens observadas de rotas e arestas coincidiram, respectivamente, com $n!$ e $n \cdot n!$ em todas as amostras concluídas. Os tempos são apresentados em milissegundos e resumidos pela mediana; os valores individuais e a dispersão estão no apêndice.

Tabela 5: Resultados observados na CPU com todas as origens.

n	Rotas	Arestas	Amostras	Mediana (ms)
4	24	96	11	0,010319
5	120	600	11	0,022244
6	720	4.320	11	0,151294
7	5.040	35.280	11	0,882772
8	40.320	322.560	11	6,237841
9	362.880	3.265.920	11	48,536747
10	3.628.800	36.288.000	9	493,659598
11	39.916.800	439.084.800	7	5.258,574811
12	479.001.600	5.748.019.200	6	63.741,750577
13	6.227.020.800	80.951.270.400	1	700.746,492811

Para $n = 13$, a única execução concluiu em aproximadamente 11 min 41 s e encontrou custo mínimo 130, com a rota $1 \to 2 \to 10 \to 8 \to 7 \to 13 \to 5 \to 4 \to 11 \to 12 \to 3 \to 9 \to 6 \to 1$. Por ser uma única amostra, esse tempo não permite avaliar a variabilidade para $n = 13$. Para $n = 12$, a mediana das seis amostras foi aproximadamente 1 min 4 s. Não há resultado concluído sem fixação para $n = 14$ neste conjunto.

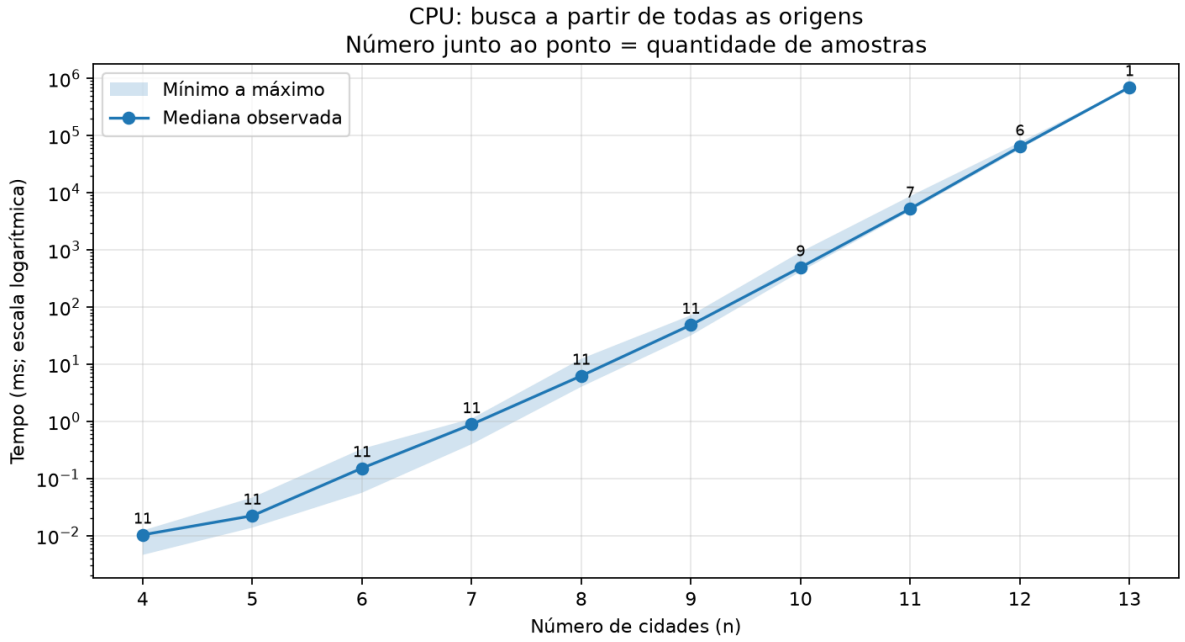


Figura 7: Tempos observados na CPU para todas as origens; a faixa mostra o mínimo e o máximo.

6.3 Custo de avaliação por rota

A avaliação percorre as n arestas do circuito, inclusive o retorno à origem. Para $n = 10$, foram avaliadas 362.880 rotas e 3.628.800 arestas ($10 \cdot 362.880 = 10!$) na CPU e em cada passagem GPU. Na CPU, a avaliação e montagem da rota custam $\Theta(n)$ por rota, e a busca completa custa $\Theta(n!)$. Na GPU, embora haja n consultas de arestas por rota, a reconstrução da permutação também percorre cidades candidatas em laços aninhados, podendo custar $\Theta(n^2)$ por rota. Logo, a contagem de arestas isoladamente não descreve todo o trabalho nem o tempo paralelo da GPU.

Portanto, o limite $\Theta(n!)$ demonstrado para a implementação sequencial **não é automaticamente um limite para o tempo da implementação CUDA**. Em uma passagem, a reconstrução e avaliação das $(n-1)!$ permutações têm custo total limitado por $O(n^2(n-1)!)$ em trabalho, além dos custos de redução e movimentação de dados. Trabalho total e tempo decorrido são medidas distintas: o tempo da GPU depende, entre outros fatores, do paralelismo disponível e da distribuição das tarefas. A segunda passagem altera o trabalho por um fator constante, mas também compõe o tempo observado.

6.4 Limiar prático da intratabilidade

Na CPU, $n = 13$ levou em mediana aproximadamente 31,84 s; o primeiro tempo acima de um minuto ocorreu em $n = 14$, com cerca de 7 min 13 s. Para $n = 15$, a única amostra disponível levou aproximadamente 1 h 39 min 41 s. Na GPU, $n = 14$ levou cerca de 6,07 s, $n = 15$ cerca de 1 min 45 s, $n = 16$ cerca de 26 min 33 s e $n = 17$ cerca de 7 h 31 min. Os tempos para os maiores tamanhos são observações de uma amostra e não devem ser tratados como estimativas estáveis; o tempo GPU inclui inicialização e transferências. Os registros das execuções estão nas Figuras 3 e 4.

O Merge Sort é um exemplo de algoritmo com complexidade $\Theta(n \log n)$, enquanto a busca exaustiva sequencial aqui estudada tem crescimento fatorial. A recorrência $T(n) = 2T(n-1) + c$ discutida na seção 4.2 do material é um exemplo distinto de crescimento $\Theta(2^n)$; ela não é a recorrência usada para deduzir o custo deste TSP. A comparação evidencia que fatores polinomiais, exponenciais e fatoriais produzem comportamentos muito diferentes conforme a entrada aumenta.

6.5 Classes P e NP e heurísticas

A classe P reúne problemas de decisão solucionáveis em tempo polinomial; NP inclui problemas de decisão cujas soluções propostas podem ser verificadas em tempo polinomial. A versão de decisão do TSP é NP-completa, enquanto sua versão de otimização é NP-difícil. Não se conhece um algoritmo polinomial exato geral para o problema, mas isso não prova que tal algoritmo não exista: a questão $P = NP$ permanece em aberto.

Uma heurística gulosa pode escolher, em cada passo, uma cidade que pareça vantajosa localmente. Essa escolha não garante a melhor combinação global e, portanto, não assegura o custo mínimo para todas as instâncias. Heurísticas e metaheurísticas podem produzir soluções úteis em tempo menor, mas sua qualidade e suas garantias dependem do método empregado.

Como contraexemplo, considere a matriz simétrica abaixo e a heurística do vizinho

mais próximo, iniciada em A:

	A	B	C	D
A	0	2	3	3
B	2	0	3	3
C	3	3	0	10
D	3	3	10	0

A heurística escolhe primeiro B, a cidade mais próxima de A. De B, há empate entre C e D; escolhendo C, obtém-se $A \to B \to C \to D \to A$, de custo $2 + 3 + 10 + 3 = 18$. Se o desempate escolher D, a rota inversa também custa 18. Entretanto, $A \to C \to B \to D \to A$ custa $3 + 3 + 3 + 3 = 12$. Enumerando as seis rotas com origem em A, verifica-se que o custo mínimo é 12. Assim, nesse exemplo a escolha gulosa não encontra uma rota ótima, independentemente do desempate mencionado.

7 Conclusão

O estudo confirmou a enumeração de $(n-1)!$ rotas e a avaliação de $n!$ arestas por passagem. Na GPU, o tempo medido inclui duas passagens, enquanto as contagens apresentadas correspondem à primeira. A implementação CPU concluiu até $n = 15$ e a GPU até $n = 17$, mas ambas apresentaram crescimento acentuado do tempo, ilustrando a intratabilidade prática da busca exaustiva. A GPU concluiu instâncias maiores no equipamento testado, mas os ambientes distintos, o número desigual de amostras e os custos de inicialização e transferência impedem uma comparação rigorosa de desempenho. As medianas agregadas são mais informativas que uma amostra isolada quando há repetições; onde só há uma execução, a limitação foi mantida explícita. Uma avaliação futura deve repetir cada tamanho várias vezes e, para comparar CPU e GPU, usar o mesmo computador e as mesmas condições.

A Tempos individuais e dispersão

A.1 Medições individuais: CPU, origem fixa

Tempos em milissegundos. Cada coluna corresponde a uma execução; o traço indica que não há medição para aquele tamanho.

n	A	B	C	D
3	0,000630	0,006473	0,005932	0,000701
4	0,003730	0,001112	0,000481	0,000471
5	0,001180	0,002365	0,001683	0,001052
6	0,005430	0,007114	0,007415	0,004709
7	0,048309	0,057766	0,035712	0,026098
8	0,384213	0,793352	0,263428	0,422930
9	3,575375	3,311486	4,599842	2,469023
10	21,730290	23,454852	24,148688	20,703818
11	–	241,149517	234,589688	220,352922
12	–	2816,919732	2675,783333	2467,607679
13	–	32885,400771	–	30599,227790
14	–	450664,889607	–	419840,804168
15	–	–	–	5981114,958103

n	E	F
3	0,000902	0,000942
4	0,000582	0,000511
5	0,001042	0,001183
6	0,005099	0,005109
7	0,027983	0,027643
8	0,197861	0,203407
9	2,937508	2,379585
10	20,937891	22,507273
11	241,903932	213,259329
12	2690,764910	2529,056042
13	32980,531361	30787,241502
14	444462,081083	421648,064272

Dispersão dos tempos (ms; três algarismos significativos):

n	Amostras	Mínimo	Mediana	Máximo
3	6	0.00063	0.000922	0.00647
4	6	0.000471	0.000547	0.00373
5	6	0.00104	0.00118	0.00236
6	6	0.00471	0.00527	0.00741
7	6	0.0261	0.0318	0.0578
8	6	0.198	0.324	0.793
9	6	2.38	3.12	4.6
10	6	20.7	22.1	24.1
11	5	213	235	242

n	Amostras	Mínimo	Mediana	Máximo
12	5	2.47e+03	2.68e+03	2.82e+03
13	4	3.06e+04	3.18e+04	3.3e+04
14	4	4.2e+05	4.33e+05	4.51e+05
15	1	5.98e+06	5.98e+06	5.98e+06

Contagens registradas (rotas e arestas):

n	Avaliações de rotas	Avaliações de arestas
3	2	6
4	6	24
5	24	120
6	120	720
7	720	5040
8	5040	40320
9	40320	362880
10	362880	3628800
11	3628800	39916800
12	39916800	479001600
13	479001600	6227020800
14	6227020800	87178291200
15	87178291200	1307674368000

A.2 Medições individuais: GPU, origem fixa

Tempos em milissegundos. Cada coluna corresponde a uma execução; o traço indica que não há medição para aquele tamanho.

n	A	B
3	161,061415	142,841025
4	0,065130	0,357791
5	0,067490	1,408151
6	0,061299	0,635886
7	0,065350	0,302767
8	0,063500	0,503650
9	0,102229	0,355900
10	0,568498	0,575293
11	3,807393	2,853125
12	40,446765	24,476475
13	493,333829	311,104639
14	7256,399898	4890,258530
15	116386,010315	93118,212972
16	1622276,604931	1563253,068073
17	27074814,512025	–

Dispersão dos tempos (ms; três algarismos significativos):

n	Amostras	Mínimo	Mediana	Máximo
3	2	143	152	161
4	2	0.0651	0.211	0.358
5	2	0.0675	0.738	1.41
6	2	0.0613	0.349	0.636
7	2	0.0654	0.184	0.303
8	2	0.0635	0.284	0.504
9	2	0.102	0.229	0.356
10	2	0.568	0.572	0.575
11	2	2.85	3.33	3.81
12	2	24.5	32.5	40.4
13	2	311	402	493
14	2	4.89e+03	6.07e+03	7.26e+03
15	2	9.31e+04	1.05e+05	1.16e+05
16	2	1.56e+06	1.59e+06	1.62e+06
17	1	2.71e+07	2.71e+07	2.71e+07

Contagens registradas (rotas e arestas):

n	Avaliações de rotas	Avaliações de arestas
3	2	6
4	6	24
5	24	120
6	120	720
7	720	5040
8	5040	40320
9	40320	362880
10	362880	3628800
11	3628800	39916800
12	39916800	479001600
13	479001600	6227020800
14	6227020800	87178291200
15	87178291200	1307674368000
16	1307674368000	20922789888000
17	20922789888000	355687428096000

A.3 Medições individuais: CPU, todas as origens

Tempos em milissegundos. Cada coluna corresponde a uma execução; o traço indica que não há medição para aquele tamanho.

n	A	B	C	D
4	0,008351	0,010319	–	0,012271
5	0,014735	0,017781	–	0,046310
6	0,084926	0,151294	–	0,187042
7	0,537665	0,882772	–	1,052807

n	A	B	C	D
8	4,486224	7,562803	–	12,416778
9	34,449002	54,860108	–	71,813808
10	461,714364	–	–	749,880140
11	4772,838389	–	–	8703,860762
12	63138,217306	–	73979,988669	–

n	E	F	G	H
4	0,010425	0,009403	0,011913	0,011763
5	0,035964	0,018698	0,038576	0,038172
6	0,175412	0,139499	0,156110	0,165718
7	0,929429	0,899237	1,023187	1,112638
8	6,237841	12,228080	9,509803	8,808975
9	60,551169	48,536747	60,727494	59,518269
10	917,776069	–	715,030039	707,644363
11	–	–	6426,228867	–

n	I	J	K	L
4	0,005236	0,004597	0,008692	0,011325
5	0,017003	0,013791	0,022244	0,027418
6	0,095747	0,056287	0,082597	0,330597
7	0,640153	0,398442	0,545165	0,612495
8	5,612293	4,021640	4,470285	5,237100
9	40,196038	31,860192	37,573067	46,067216
10	417,484334	450,866226	464,730022	493,659598
11	5994,823967	4863,752327	4937,410892	5258,574811
12	76028,134951	58008,567679	64345,283848	57669,253561
13	–	700746,492811	–	–

Dispersão dos tempos (ms; três algarismos significativos):

n	Amostras	Mínimo	Mediana	Máximo
4	11	0.0046	0.0103	0.0123
5	11	0.0138	0.0222	0.0463
6	11	0.0563	0.151	0.331
7	11	0.398	0.883	1.11
8	11	4.02	6.24	12.4
9	11	31.9	48.5	71.8
10	9	417	494	918
11	7	4.77e+03	5.26e+03	8.7e+03
12	6	5.77e+04	6.37e+04	7.6e+04
13	1	7.01e+05	7.01e+05	7.01e+05

Contagens registradas (rotas e arestas):

n	Avaliações de rotas	Avaliações de arestas
4	24	96
5	120	600
6	720	4320
7	5040	35280
8	40320	322560
9	362880	3265920
10	3628800	36288000
11	39916800	439084800
12	479001600	5748019200
13	6227020800	80951270400

Referências

[1] UNIVERSIDADE DE VILA VELHA. *Projeto e Análise de Algoritmos: Unidade 4*. Material da disciplina, seções 4.1 e 4.2.

[2] *Proposta de Atividade Investigativa: Explorando a Intratabilidade e a Força Bruta no Problema do Caixeiro Viajante (TSP)*. Documento fornecido para a atividade.

[3] CABRITTO CORPS. *tsp-go: implementação do Problema do Caixeiro Viajante*. Repositório de código-fonte. Disponível em: `https://github.com/Cabritto-Corps/tsp-go`.